

LAPORAN PRAKTIKUM

SISTEM OPERASI

Dibuat untuk Memenuhi Tugas Mata Kuliah Sistem Operasi



Dosen Pengampu Mata Kuliah:

Triawan Adi Cahyanto, M.K

Disusun Oleh Kelompok 3:

Muhamad Gufron (2410651053)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

TAHUN AKADEMIK 2024/2025

KATA PENGANTAR

Puji syukur kehadiran Allah SWT yang telah memberikan rahmat dan hidayah-Nya sehingga kami dapat menyelesaikan tugas makalah yang berjudul “Memori Dalam Komputer” ini tepat waktu.

Adapun tujuan dari penulisan makalah ini adalah untuk memenuhi tugas dosen pada mata kuliah Dasar-Dasar Sistem. Selain itu, makalah ini juga bertujuan untuk menambah wawasan tentang Kerja Sama Kelompok bagi para pembaca dan juga bagi penulis.

Kami mengucapkan terima kasih kepada ibu Nur Qodariyah Fitriyah, S selaku dosen mata kuliah Pengembangan Diri yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan bidang studi yang kami tekuni.

Kami juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga kami dapat menyelesaikan makalah ini.

Kami menyadari, makalah yang kami tulis ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun akan kami nantikan demi kesempurnaan makalah ini.

BAB I

PENDAHULUAN

1.2 Latar Belakang

Sistem operasi merupakan perangkat lunak utama yang mengatur seluruh aktivitas komputer, mulai dari pengelolaan perangkat keras, proses, memori, hingga interaksi dengan pengguna. Pemahaman tentang sistem operasi sangat penting bagi mahasiswa teknik informatika agar mampu memahami cara kerja komputer secara menyeluruh dan efisien.

1.2 Tujuan

Tujuan dari praktikum ini adalah untuk:

1. Memahami konsep dasar sistem operasi dan fungsinya.
2. Mempelajari cara kerja sistem operasi dalam mengelola proses, file, dan memori.
3. Mengetahui penggunaan **command line** serta **system call** dalam sistem Linux.
4. Mengetahui penerapan langsung konsep sistem operasi melalui praktik pemrograman C dan pengujian layanan sistem.

1.3 Manfaat

Melalui praktikum ini, mahasiswa diharapkan dapat:

1. Memahami hubungan antara perangkat keras, kernel, dan aplikasi.
2. Mampu menggunakan perintah dasar Linux untuk mengelola sistem.

BAB II

PEMBAHASAN

Pada praktikum Sistem Operasi ini, mahasiswa mempelajari cara kerja dasar sistem operasi melalui berbagai kegiatan di Linux. Praktikum meliputi penggunaan Command Line Interface (CLI), percobaan dengan system call, serta pengelolaan file dan proses menggunakan bahasa C.

Mahasiswa juga belajar mengelola layanan sistem (systemd), memahami konsep linking dan library, serta membuat kernel module sederhana untuk menambah fungsi kernel. Selain itu, diperkenalkan pula struktur sistem operasi, mulai dari perangkat keras hingga aplikasi pengguna.

Secara keseluruhan, praktikum ini membantu mahasiswa memahami bagaimana sistem operasi bekerja dalam mengelola sumber daya komputer dan menjalankan proses secara efisien.

1.1 Menjelajahi Layanan OS

Lihat Informasi Sistem

```
gufro@MSI:~$ uname -a
Linux MSI 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
gufro@MSI:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.3 LTS
Release:        24.04
```

Lihat proses yang berjalan

```
gufro@MSI:~$ ps aux | head -20
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root             1   0.8  0.1 21796 12148 ?        Ss   14:43   0:00 /sbin/init
root             2   0.0  0.0   3872  1664 ?        Sl   14:43   0:00 /init
root             6   0.0  0.0   3872  1792 ?        Sl   14:43   0:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10
--log-truncate
root            42   0.3  0.1  50428 15232 ?        S<s  14:43   0:00 /usr/lib/systemd/systemd-journald
root            90   0.1  0.0  25264  6016 ?        Ss   14:43   0:00 /usr/lib/systemd/systemd-udevd
systemd+      104   0.1  0.1  21456 12160 ?        Ss   14:43   0:00 /usr/lib/systemd/systemd-resolved
systemd+      105   0.0  0.0   91024  7552 ?        Ssl  14:43   0:00 /usr/lib/systemd/systemd-timesyncd
root          165   0.0  0.0   4236  2560 ?        Ss   14:43   0:00 /usr/sbin/cron -f -P
message+      166   0.0  0.0   9636  4608 ?        Ss   14:43   0:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --sy
stemd-activation --syslog-only
root          173   0.1  0.1  17968  8448 ?        Ss   14:43   0:00 /usr/lib/systemd/systemd-logind
root          175   0.1  0.1 1756096 12288 ?        Ssl  14:43   0:00 /usr/libexec/wsl-pro-service -vv
root          189   0.0  0.0   3160  1920 hvc0   Ss+  14:43   0:00 /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600
vt220
root          197   0.0  0.0   3116  1792 tty1    Ss+  14:43   0:00 /sbin/agetty -o -p -- \u --noclear - linux
syslog        199   0.1  0.0  222508  5376 ?        Ssl  14:43   0:00 /usr/sbin/rsyslogd -n -iNONE
root          206   0.2  0.2 107032 21888 ?        Ssl  14:43   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade
--shutdown --wait-for-signal
root          291   0.0  0.0   3076   896 ?        Ss   14:43   0:00 /init
root          292   0.0  0.0   3092  1152 ?        S   14:43   0:00 /init
gufro         293   0.0  0.0   6204  5248 pts/0   Ss   14:43   0:00 -bash
root          294   0.0  0.0   6820  4608 pts/1   Ss   14:43   0:00 /bin/login -f
```

Lihat Penggunaan Memori

```
gufro@MSI:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:           7.6Gi       479Mi       6.9Gi       3.5Mi       424Mi       7.2Gi
Swap:          2.0Gi          0B       2.0Gi
```

Lihat Informasi CPU

```
gufro@MSI:~$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 12
On-line CPU(s) list:   0-11
Vendor ID:              GenuineIntel
Model name:             Intel(R) Core(TM) 5 120U
CPU family:             6
Model:                 186
Thread(s) per core:    2
Core(s) per socket:    6
Socket(s):              1
Stepping:               3
BogoMIPS:               4992.00
Flags:                  fpu vae de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht sys
call nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx
ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx fl6c rdrand hypervisor lahf_lm abm 3dnowprefetch
ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid rdseed adx
smap clflushopt clwb sha_ni xsaveopt xsaves avx_vnni vnni umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_clear s
erialize flush_lld arch_capabilities

Virtualization features:
Virtualization:        VT-x
Hypervisor vendor:     Microsoft
Virtualization type:   full
Caches (sum of all):
```

```

Virtualization features:
  Virtualization:      VT-x
  Hypervisor vendor:    Microsoft
  Virtualization type:  full
Caches (sum of all):
  L1d:                 288 KiB (6 instances)
  L1i:                 192 KiB (6 instances)
  L2:                  7.5 MiB (6 instances)
  L3:                  12 MiB (1 instance)
NUMA:
  NUMA node(s):        1
  NUMA node0 CPU(s):   0-11
Vulnerabilities:
  Gather data sampling: Not affected
  Itlb multihit:        Not affected
  L1tf:                 Not affected
  Mds:                  Not affected
  Meltdown:             Not affected
  Mmio stale data:      Not affected
  Reg file data sampling: Mitigation; Clear Register File
  Retbleed:             Mitigation; Enhanced IBRS
  Spec rstack overflow: Not affected
  Spec store bypass:    Mitigation; Speculative Store Bypass disabled via prctl
  Spectre v1:           Mitigation; usercopy/swapgs barriers and __user pointer sanitization
  Spectre v2:           Mitigation; Enhanced / Automatic IBRS; IBPB conditional; RSB filling; PBRSB-eIBRS SW sequence; BHI BHI_DIS_
  Srbds:                Not affected
  Tsx async abort:      Not affected

```

Lihat Perangkat yang terpasang

```

gufro@MSI:~$ lsblk
NAME                                MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda      8:0      0 388.4M  1 disk
sdb      8:16     0   186M  1 disk
sdc      8:32     0     2G   0 disk [SWAP]
sdd      8:48     0     1T   0 disk /mnt/wslg/distro
/

```

1.2 Praktik Program C – Menggunakan Layanan OS

Program `os_services.c` menunjukkan bagaimana sistem operasi menyediakan berbagai layanan penting:

1. **Program Execution Service** – Menampilkan ID proses (`getpid`) dan ID proses induk (`getppid`).
2. **I/O Operations Service** – Membuat dan menulis file dengan fungsi `fopen` dan `fprintf`.
3. **File System Manipulation** – Mengecek ukuran dan izin file menggunakan `stat`.
4. **Communications** – Menampilkan nama host sistem lewat `getenv("HOSTNAME")`.

5. **Error Detection** – Menangani kesalahan saat membuka file yang tidak ada dengan perror.
6. **Resource Allocation** – Mengalokasikan dan membebaskan memori menggunakan malloc dan free.

Program ini memperlihatkan bagaimana OS berperan dalam menjalankan, mengatur, dan mengawasi sumber daya sistem komputer.

```
gufro@MSI:~$ os_services.c
os_services.c: command not found
gufro@MSI:~$ nano os_services.c
gufro@MSI:~$ gcc -o os_services os_services.c
gufro@MSI:~$ ./os_services
=== Program Execution Service ===
Process ID (PID): 483
Parent Process ID (PPID): 293

=== I/O Operations Service ===
File created and written successfully

=== File System Manipulation ===
File size: 23 bytes
File permissions: 644

=== Communications ===
Hostname: (null)

=== Error Detection ===
Error opening file: No such file or directory

=== Resource Allocation ===
Memory allocated successfully
```

2.1 Command Line Interface (CLI)

Praktik ini menunjukkan bagaimana pengguna berinteraksi dengan sistem operasi melalui **Command Line Interface (CLI)** di Linux. Perintah seperti pwd, ls, dan cd digunakan untuk navigasi direktori. Fitur **pipelining** (|) menghubungkan output antarperintah, **redirection** (>, >>, <) mengatur input-output file, dan **background**

process (&, jobs, fg) mengelola proses berjalan di latar belakang dengan efisien dan fleksibel.

Navigasi dasar

```
gufro@MSI:~$ ls -la
total 64
drwxr-x--- 5 gufro gufro 4096 Oct 14 14:54 .
drwxr-xr-x 3 root root 4096 Oct 8 10:45 ..
-rw----- 1 gufro gufro 935 Oct 13 23:59 .bash_history
-rw-r--r-- 1 gufro gufro 220 Oct 8 10:45 .bash_logout
-rw-r--r-- 1 gufro gufro 3771 Oct 8 10:45 .bashrc
drwx----- 2 gufro gufro 4096 Oct 8 10:49 .cache
drwxr-xr-x 3 gufro gufro 4096 Oct 13 23:31 .local
-rw-rw-r-- 1 gufro gufro 0 Oct 14 14:43 .motd_shown
-rw-r--r-- 1 gufro gufro 807 Oct 8 10:45 .profile
-rw-r--r-- 1 gufro gufro 0 Oct 13 23:25 .sudo_as_admin_successful
-rwxr-xr-x 1 gufro gufro 16488 Oct 14 14:54 os_services
-rw-r--r-- 1 gufro gufro 1358 Oct 14 14:54 os_services.c
drwxr-xr-x 2 gufro gufro 4096 Oct 13 23:38 process_manager
-rw-r--r-- 1 gufro gufro 23 Oct 14 14:54 test_file.txt
gufro@MSI:~$ cd /tmp
gufro@MSI:/tmp$ mkdir test_dir
gufro@MSI:/tmp$
```

Pipelining

```
gufro@MSI:/tmp$ ls -l | grep "^d"
drwx----- 2 root root 4096 Oct 14 14:43 snap-private-tmp
drwx----- 3 root root 4096 Oct 14 14:43 systemd-private-5e12104b87d946829d3853c9e453ccac-systemd-logind.service-NoiiTa
drwx----- 3 root root 4096 Oct 14 14:43 systemd-private-5e12104b87d946829d3853c9e453ccac-systemd-resolved.service-tjxUms
drwx----- 3 root root 4096 Oct 14 14:43 systemd-private-5e12104b87d946829d3853c9e453ccac-systemd-timesyncd.service-W8fxzg
drwx----- 3 root root 4096 Oct 14 14:43 systemd-private-5e12104b87d946829d3853c9e453ccac-wsl-pro.service-la70ag
drwxr-xr-x 2 gufro gufro 4096 Oct 14 14:57 test_dir
gufro@MSI:/tmp$ ps aux | grep bash | wc -l
3
gufro@MSI:/tmp$
```

Redirection


```

gufro@MSI:/tmp$ echo "Hello World" > output.txt
gufro@MSI:/tmp$ cat output.txt
Hello World
gufro@MSI:/tmp$ cat output.txt >> output.txt
cat: output.txt: input file is output file
gufro@MSI:/tmp$ cat < output.txt
Hello World

```

Background processes

```

gufro@MSI:/tmp$ sleep 30 &
[1] 505
gufro@MSI:/tmp$ jobs
[1]+  Running                  sleep 30 &
gufro@MSI:/tmp$ fg %1
sleep 30

```

2.2 Program C - Shell Sederhana

Program **Shell Sederhana** pada Linux berfungsi untuk meniru cara kerja terminal menggunakan bahasa C. Program ini menampilkan prompt myshell>, membaca perintah pengguna, lalu mengeksekusinya dengan `execvp()`. Proses baru dibuat menggunakan `fork()`, dan proses induk menunggu anak selesai dengan `wait()`. Ketik `exit` untuk keluar..

#Buat file simple_shell.c

```
gufro@MSI:/tmp$ nano simple_shell.c
gufro@MSI:/tmp$ gcc -o simple_shell simple_shell.c
gufro@MSI:/tmp$ ./simple_shell
myshell>
myshell> echo Hallo word
Hallo word
myshell> ^Z
[1]+  Stopped                  ./simple_shell
```

3.1 Process Control System Calls

Program **Process Control System Calls** digunakan untuk mempelajari cara sistem operasi Linux mengelola proses. Dalam praktik ini, fungsi `fork()` digunakan untuk membuat proses anak dari proses induk. Proses anak menjalankan perintah `ls -l` dengan `execlp()`, sementara proses induk menunggu anak selesai menggunakan `wait()`. Fungsi `getpid()` dan `getppid()` menampilkan ID proses induk dan anak. Program ini membantu memahami bagaimana proses dibuat, dijalankan, dan disinkronkan di Linux melalui system call dasar seperti `fork()`, `exec()`, dan `wait()`.

Buat file process_syscalls.c

```

gufro@MSI:/tmp$ nano process_syscalls.c
gufro@MSI:/tmp$ gcc -o process_syscalls.c
gcc: fatal error: no input files
compilation terminated.
gufro@MSI:/tmp$ gcc -o process_syscalls process_syscalls.c
gufro@MSI:/tmp$ ./process_syscalls
=== Process Control System Calls ===
Parent PID: 561
Parent waiting for child (PID: 562)...
Child PID: 562
Child's Parent PID: 561
Child executing 'ls' command...
total 72
-rw-r--r-- 1 gufro gufro 12 Oct 14 15:03 output.txt
-rwxr-xr-x 1 gufro gufro 16360 Oct 14 15:24 process_syscalls
-rw-r--r-- 1 gufro gufro 995 Oct 14 15:21 process_syscalls.c
-rwxr-xr-x 1 gufro gufro 16528 Oct 14 15:14 simple_shell
-rw-r--r-- 1 gufro gufro 1229 Oct 14 15:14 simple_shell.c
drwx----- 2 root root 4096 Oct 14 14:43 snap-private-tmp
drwx----- 3 root root 4096 Oct 14 14:43 systemd-private-5e12104b87d946829d3853c9e453ccac-systemd-logind.service-NoiiTa
drwx----- 3 root root 4096 Oct 14 14:43 systemd-private-5e12104b87d946829d3853c9e453ccac-systemd-resolved.service-tjxUms
drwx----- 3 root root 4096 Oct 14 14:43 systemd-private-5e12104b87d946829d3853c9e453ccac-systemd-timesyncd.service-W8fxzg
drwx----- 3 root root 4096 Oct 14 14:43 systemd-private-5e12104b87d946829d3853c9e453ccac-wsl-pro.service-la70ag
drwxr-xr-x 2 gufro gufro 4096 Oct 14 14:57 test_dir
Child exited with status: 0

```

3.2 File Management System Calls

Program **File Management System Calls** digunakan untuk mempelajari cara Linux mengelola file. Program ini memakai `open()` untuk membuat atau membuka file, `write()` untuk menulis data, `read()` untuk membaca isi file, dan `close()` untuk menutup file. File juga bisa dihapus dengan `unlink()`. anak selesai, status keluar akan ditampilkan oleh proses induk. Melalui program ini, pengguna dapat memahami konsep dasar pembentukan, eksekusi, dan sinkronisasi proses pada sistem operasi Linux.

#Buat file `file_syscalls.c`:

```

gufro@MSI:/tmp$ nano file_syscalls.c
gufro@MSI:/tmp$ gcc -o file_syscalls file_syscalls.c
gufro@MSI:/tmp$ ./file_syscalls
=== File Management System Calls ===
File descriptor: 3
Bytes written: 25
File closed
Read from file: Hello from system calls!

```

3.3 Information Maintenance System Calls

Program **Information Maintenance System Calls** digunakan untuk menampilkan informasi sistem dan proses di Linux. Program ini menampilkan ID proses, ID pengguna, waktu saat ini, serta jumlah *clock ticks* sejak sistem dijalankan. Fungsi `sleep()` digunakan untuk menunda proses selama dua detik sebelum melanjutkan kembali.

#Buat file `info_syscalls.c`

```
gufro@MSI:/tmp$ nano info_syscalls.c
gufro@MSI:/tmp$ gcc -o info_syscalls info_syscalls.c
gufro@MSI:/tmp$ ./info_syscalls
=== Information Maintenance System Calls ===

Process ID: 617
Parent Process ID: 293
User ID: 1000
Group ID: 1000
Current time: Tue Oct 14 15:32:57 2025
Clock ticks since boot: 1718270372
Sleeping for 2 seconds...
Awake!
```

4.1 Menggunakan system

Pada **praktik System Services** ini, pengguna mempelajari cara mengelola layanan (service) di Linux menggunakan **systemd**, yaitu sistem inisialisasi modern yang mengatur proses saat sistem dinyalakan, dijalankan, dan dimatikan. Perintah `systemctl list-units --type=service` digunakan untuk melihat semua layanan yang aktif di sistem. Untuk mengetahui status layanan tertentu, seperti SSH, digunakan perintah `systemctl status ssh`, yang menampilkan informasi detail termasuk kondisi, waktu aktif, dan log aktivitas. Jika ingin melihat riwayat log layanan, perintah `journalctl -u ssh -n 20` menampilkan 20 log terakhir dari layanan SSH. Selain itu, `systemctl list-unit-files --type=service --state=enabled` digunakan untuk melihat layanan apa saja yang berjalan

otomatis saat sistem melakukan boot. Melalui praktik ini, pengguna memahami cara memonitor, mengontrol, dan menganalisis layanan sistem menggunakan systemd.

#Lihat semua service

```
gufro@MSI:/tmp$ systemctl list-units --type=service
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
console-getty.service              loaded active running Console Getty
console-setup.service              loaded active exited Set console font and keymap
cron.service                       loaded active running Regular background program processing daemon
dbus.service                       loaded active running D-Bus System Message Bus
getty@tty1.service                 loaded active running Getty on tty1
keyboard-setup.service             loaded active exited Set the console keyboard layout
kmod-static-nodes.service          loaded active exited Create List of Static Device Nodes
rsyslog.service                   loaded active running System Logging Service
setvtrgb.service                  loaded active exited Set console scheme
snapd.seeded.service              loaded active exited Wait until snapd is fully seeded
systemd-binfmt.service            loaded active exited Set Up Additional Binary Formats
systemd-journal-flush.service      loaded active exited Flush Journal to Persistent Storage
systemd-journald.service           loaded active running Journal Service
systemd-logind.service             loaded active running User Login Management
systemd-modules-load.service       loaded active exited Load Kernel Modules
systemd-remount-fs.service         loaded active exited Remount Root and Kernel File Systems
systemd-resolved.service           loaded active running Network Name Resolution
systemd-sysctl.service            loaded active exited Apply Kernel Variables
systemd-timesyncd.service          loaded active running Network Time Synchronization
systemd-tmpfiles-setup-dev-early.service loaded active exited Create Static Device Nodes in /dev gracefully
systemd-tmpfiles-setup-dev.service loaded active exited Create Static Device Nodes in /dev
systemd-tmpfiles-setup.service     loaded active exited Create Volatile Files and Directories
systemd-udev-trigger.service       loaded active exited Coldplug All udev Devices
systemd-udevd.service              loaded active running Rule-based Manager for Device Events and Files
systemd-update-utmp.service        loaded active exited Record System Boot/Shutdown in UTMP
systemd-user-sessions.service      loaded active exited Permit User Sessions
unattended-upgrades.service        loaded active running Unattended Upgrades Shutdown
user-runtime-dir@1000.service      loaded active exited User Runtime Directory /run/user/1000
user@1000.service                 loaded active running User Manager for UID 1000
wsl-pro.service                   loaded active running Bridge to Ubuntu Pro agent on Windows

Legend: LOAD    → Reflects whether the unit definition was properly loaded.
         ACTIVE → The high-level unit activation state, i.e. generalization of SUB.
         SUB    → The low-level unit activation state, values depend on unit type.
```

Status service

```
gufro@MSI:/tmp$ systemctl status ssh
○ ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: inactive (dead)
TriggeredBy: ● ssh.socket
   Docs: man:sshd(8)
        man:sshd_config(5)
```

Lihat log service

```
gufro@MSI:/tmp$ journalctl -u ssh -n 20
-- No entries --
gufro@MSI:/tmp$ |
```

Lihat service yang berjalan saat boot

```
gufro@MSI:/tmp$ systemctl list-unit-files --type=service --state=enabled
UNIT FILE                                STATE    PRESET
apparmor.service                        enabled  enabled
apport.service                          enabled  enabled
cloud-config.service                    enabled  enabled
cloud-final.service                      enabled  enabled
cloud-init-local.service                 enabled  enabled
cloud-init.service                       enabled  enabled
console-setup.service                    enabled  enabled
cron.service                            enabled  enabled
dmesg.service                            enabled  enabled
e2scrub_reap.service                     enabled  enabled
getty@.service                           enabled  enabled
keyboard-setup.service                   enabled  enabled
landscape-client.service                  enabled  enabled
networkd-dispatcher.service              enabled  enabled
rsyslog.service                          enabled  enabled
setvtrgb.service                         enabled  enabled
snapd.apparmor.service                   enabled  enabled
snapd.autoimport.service                 enabled  enabled
snapd.core-fixup.service                  enabled  enabled
snapd.recovery-chooser-trigger.service    enabled  enabled
snapd.seeded.service                     enabled  enabled
snapd.service                            enabled  enabled
snapd.system-shutdown.service             enabled  enabled
systemd-pstore.service                   enabled  enabled
systemd-resolved.service                  enabled  enabled
systemd-timesyncd.service                 enabled  enabled
ua-reboot-cmds.service                    enabled  enabled
ubuntu-advantage.service                  enabled  enabled
unattended-upgrades.service               enabled  enabled
wsl-pro.service                           enabled  enabled
30 unit files listed.
```

4.2 Membuat Custom System Service

mengelola layanan (service) sendiri di Linux menggunakan **systemd**. Program `my_daemon.c` berfungsi sebagai *daemon*, yaitu proses yang berjalan di latar belakang tanpa interaksi langsung dengan pengguna. Program ini menggunakan fungsi `fork()` untuk membuat proses baru, `setsid()` untuk memisahkan diri dari terminal, serta `syslog()` untuk mencatat aktivitas ke log sistem. Saat dijalankan, daemon menampilkan pesan status setiap 30 detik melalui `syslog` dan dapat dihentikan menggunakan sinyal `SIGTERM`. Setelah program dikompilasi, file service dibuat di direktori

/etc/systemd/system/ dengan konfigurasi yang menentukan cara daemon dijalankan dan diatur. Perintah systemctl daemon-reload digunakan untuk memuat ulang konfigurasi, systemctl start my_daemon untuk memulai service, dan systemctl status my_daemon untuk memeriksa statusnya. Log aktivitas daemon dapat dilihat menggunakan journalctl -u my_daemon. Melalui praktik ini, pengguna memahami cara membuat, menjalankan, dan memantau layanan khusus menggunakan systemd di Linux.

#Buat program sederhana my_daemon.c

```
gufro@MSI:/tmp$ nano my_daemon.c
gufro@MSI:/tmp$ gcc -o my_daemon my_daemon.c
gufro@MSI:/tmp$ ./my_daemon
gufro@MSI:/tmp$ |
```

```
gufro@MSI:/tmp$ /etc/systemd/system/my_daemon.service
-bash: /etc/systemd/system/my_daemon.service: No such file or directory
gufro@MSI:/tmp$ sudo nano /etc/systemd/system/my_daemon.service
[sudo] password for gufro:
gufro@MSI:/tmp$ sudo systemctl daemon-reload
gufro@MSI:/tmp$ sudo systemctl start my_daemon
Job for my_daemon.service failed because the control process exited with error code.
See "systemctl status my_daemon.service" and "journalctl -xeu my_daemon.service" for details.
gufro@MSI:/tmp$ sudo systemctl status my_daemon
* my_daemon.service - My Custom Daemon
   Loaded: loaded (/etc/systemd/system/my_daemon.service; disabled; preset: enabled)
   Active: failed (Result: exit-code) since Tue 2025-10-14 16:11:04 WIB; 26s ago
     Process: 1453 ExecStart=/path/to/my_daemon (code=exited, status=203/EXEC)
    CPU: 1ms

Oct 14 16:11:04 MSI systemd[1]: my_daemon.service: Scheduled restart job, restart counter is at 5.
Oct 14 16:11:04 MSI systemd[1]: my_daemon.service: Start request repeated too quickly.
Oct 14 16:11:04 MSI systemd[1]: my_daemon.service: Failed with result 'exit-code'.
Oct 14 16:11:04 MSI systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
gufro@MSI:/tmp$ sudo journalctl -u my_daemon
Oct 14 16:11:02 MSI systemd[1]: Starting my_daemon.service - My Custom Daemon...
Oct 14 16:11:02 MSI systemd[1]: my_daemon.service: Control process exited, code=exited, status=203/EXEC
Oct 14 16:11:02 MSI systemd[1]: my_daemon.service: Failed with result 'exit-code'.
Oct 14 16:11:02 MSI systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
Oct 14 16:11:03 MSI systemd[1]: my_daemon.service: Scheduled restart job, restart counter is at 1.
Oct 14 16:11:03 MSI systemd[1]: Starting my_daemon.service - My Custom Daemon...
Oct 14 16:11:03 MSI systemd[1]: my_daemon.service: Control process exited, code=exited, status=203/EXEC
Oct 14 16:11:03 MSI systemd[1]: my_daemon.service: Failed with result 'exit-code'.
Oct 14 16:11:03 MSI systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
Oct 14 16:11:03 MSI systemd[1]: my_daemon.service: Scheduled restart job, restart counter is at 2.
Oct 14 16:11:03 MSI systemd[1]: Starting my_daemon.service - My Custom Daemon...
Oct 14 16:11:03 MSI systemd[1]: my_daemon.service: Control process exited, code=exited, status=203/EXEC
Oct 14 16:11:03 MSI systemd[1]: my_daemon.service: Failed with result 'exit-code'.
Oct 14 16:11:03 MSI systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
Oct 14 16:11:03 MSI systemd[1]: my_daemon.service: Scheduled restart job, restart counter is at 3.
Oct 14 16:11:03 MSI systemd[1]: Starting my_daemon.service - My Custom Daemon...
```

```

Oct 14 16:11:03 MSI systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
Oct 14 16:11:03 MSI systemd[1]: my_daemon.service: Scheduled restart job, restart counter is at 3.
Oct 14 16:11:03 MSI systemd[1]: Starting my_daemon.service - My Custom Daemon...
Oct 14 16:11:03 MSI systemd[1]: my_daemon.service: Control process exited, code=exited, status=203/EXEC
Oct 14 16:11:03 MSI systemd[1]: my_daemon.service: Failed with result 'exit-code'.
Oct 14 16:11:03 MSI systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
Oct 14 16:11:03 MSI systemd[1]: my_daemon.service: Scheduled restart job, restart counter is at 4.
Oct 14 16:11:03 MSI systemd[1]: Starting my_daemon.service - My Custom Daemon...
Oct 14 16:11:03 MSI systemd[1]: my_daemon.service: Control process exited, code=exited, status=203/EXEC
Oct 14 16:11:03 MSI systemd[1]: my_daemon.service: Failed with result 'exit-code'.
Oct 14 16:11:03 MSI systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
Oct 14 16:11:04 MSI systemd[1]: my_daemon.service: Scheduled restart job, restart counter is at 5.
Oct 14 16:11:04 MSI systemd[1]: my_daemon.service: Start request repeated too quickly.
Oct 14 16:11:04 MSI systemd[1]: my_daemon.service: Failed with result 'exit-code'.
Oct 14 16:11:04 MSI systemd[1]: Failed to start my_daemon.service - My Custom Daemon.
gufro@MSI:/tmp$ sudo systemctl stop my_daemon

```

```

gufro@MSI:/tmp$ sudo systemctl stop my_daemon
gufro@MSI:/tmp$ nano math_ops.h
gufro@MSI:/tmp$ |

```

5.1 Proses Linking

Dalam Linux, **linker** dan **loader** berperan penting untuk membuat dan menjalankan program. Pada praktik ini, dibuat tiga file: `math_ops.h`, `math_ops.c`, dan `main.c`. Prosesnya dimulai dari **preprocessing** untuk menyiapkan kode, lalu **compilation** mengubahnya menjadi bahasa assembly. Tahap **assembly** menghasilkan file objek (`.o`), dan **linker** menggabungkan file tersebut menjadi satu program lengkap. Linker juga memastikan fungsi seperti `add()` dan `multiply()` terhubung dengan benar. Terakhir, **loader** memuat program ke memori agar bisa dijalankan. Proses ini menunjukkan bagaimana kode sumber berubah menjadi program yang siap dieksekusi di Linux.

Buat tiga file:

`math_ops.h`

```

gufro@MSI:/tmp$ nano math_ops.h
gufro@MSI:/tmp$ nano math_ops.c
gufro@MSI:/tmp$ |

```

```

gufro@MSI:/tmp$ nano main.c
gufro@MSI:/tmp$ |

```


1. Preprocessing

```
gufro@MSI:/tmp$ gcc -E main.c -o main.i
gufro@MSI:/tmp$ gcc -E math_ops.c -o math_ops.i
gufro@MSI:/tmp$ |
```

2. Compilation (assembly)

```
gufro@MSI:/tmp$ gcc -S main.c -o main.s
gufro@MSI:/tmp$ gcc -S math_ops.c -o math_ops.s
gufro@MSI:/tmp$ |
```

3. Assembly (object files)

```
gufro@MSI:/tmp$ gcc -c main.c -o main.o
gufro@MSI:/tmp$ gcc -c math_ops.c -o math_ops.o
gufro@MSI:/tmp$ |
```

4. Linking

```
gufro@MSI:/tmp$ gcc main.o math_ops.o -o program
gufro@MSI:/tmp$ |
```

Lihat symbols dalam object file

```

gufro@MSI:/tmp$ nm main.o
U add
0000000000000000 T main
U multiply
U printf
gufro@MSI:/tmp$ nm math_ops.o
0000000000000000 T add
0000000000000018 T multiply
gufro@MSI:/tmp$ nm program
0000000000003dc8 d _DYNAMIC
0000000000003fb8 d _GLOBAL_OFFSET_TABLE_
0000000000002000 R _IO_stdin_used
w _ITM_deregisterTMCloneTable
w _ITM_registerTMCloneTable
0000000000002148 r __FRAME_END__
000000000000201c r __GNU_EH_FRAME_HDR
0000000000004010 D __TMC_END__
000000000000038c r __abi_tag
0000000000004010 B __bss_start
w __cxa_finalize@GLIBC_2.2.5
D __data_start
0000000000001100 t __do_global_dtors_aux
0000000000003dc0 d __do_global_dtors_aux_fini_array_entry
0000000000004008 D __dso_handle
0000000000003db8 d __frame_dummy_init_array_entry
w __gmon_start__
U __libc_start_main@GLIBC_2.34
0000000000004010 D _edata
0000000000004018 B _end
00000000000011d4 T _fini
0000000000001000 T _init
0000000000001060 T _start
00000000000011a2 T add
0000000000004010 b completed.0
0000000000004000 W data_start

```

```

0000000000004010 b completed.0
0000000000004000 W data_start
0000000000001090 t deregister_tm_clones
0000000000001140 t frame_dummy
0000000000001149 T main
00000000000011ba T multiply
U printf@GLIBC_2.2.5
00000000000010c0 t register_tm_clones
gufro@MSI:/tmp$ |

```

Lihat dependencies

```

gufro@MSI:/tmp$ ldd program
linux-vdso.so.1 (0x00007ffd309a8000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x000072df32400000)
/lib64/ld-linux-x86-64.so.2 (0x000072df3270e000)
gufro@MSI:/tmp$ |

```

5.2 Static vs Dynamic Linking

membahas perbedaan antara **static linking** dan **dynamic linking** saat mengompilasi program C menggunakan GCC. Pada static linking, perintah `gcc main.c math_ops.c -o program_static -static` membuat semua library dimasukkan langsung ke dalam file program. Hasilnya, ukuran file menjadi lebih besar, tetapi program dapat

dijalankan tanpa bergantung pada library eksternal. Sementara itu, dynamic linking menggunakan perintah `gcc main.c math_ops.c -o program_dynamic` tanpa opsi `-static`, sehingga program hanya memanggil library yang ada di sistem. File hasilnya lebih kecil, namun program membutuhkan library tersebut saat dijalankan. Perintah `ls -lh` digunakan untuk membandingkan ukuran file, sedangkan `ldd` digunakan untuk melihat library yang digunakan. Program static biasanya menampilkan pesan “*not a dynamic executable*”, sedangkan program dynamic menampilkan daftar library seperti `libc.so.6`. Kesimpulannya, static linking mandiri tetapi besar, sedangkan dynamic linking kecil namun bergantung pada library sistem.

Compile sebagai shared library

```
gufro@MSI:/tmp$ gcc main.c math_ops.c -o program_static -static
gufro@MSI:/tmp$
```

Dynamic linking (default)

```
gufro@MSI:/tmp$ gcc main.c math_ops.c -o program_dynamic
gufro@MSI:/tmp$
```

Bandingkan ukuran

```
gufro@MSI:/tmp$ ls -lh program_static program_dynamic
-rwxr-xr-x 1 gufro gufro 16K Oct 14 16:24 program_dynamic
-rwxr-xr-x 1 gufro gufro 767K Oct 14 16:23 program_static
gufro@MSI:/tmp$
```

Lihat dependencies

```
gufro@MSI:/tmp$ ldd program_static
not a dynamic executable
gufro@MSI:/tmp$ ldd program_dynamic
linux-vdso.so.1 (0x00007fffd94baf000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x000075132b200000)
/lib64/ld-linux-x86-64.so.2 (0x000075132b443000)
gufro@MSI:/tmp$
```

5.3 Membuat dan Menggunakan Shared Library

membahas cara membuat dan memakai shared library di C. Pertama, file sumber dikompilasi dengan perintah `gcc -c -fPIC math_ops.c -o math_ops.o`, lalu dibuat library bersama dengan `gcc -shared -o libmathops.so math_ops.o`. Setelah itu, program utama dikompilasi menggunakan library tersebut dengan `gcc main.c -L. -lmathops -o program_shared`. Program dijalankan memakai `LD_LIBRARY_PATH=.` `./program_shared` agar sistem tahu lokasi library. Terakhir, perintah `readelf -d program_shared` digunakan untuk melihat informasi tentang library yang dipakai program.

Compile sebagai shared library

```
gufro@MSI:/tmp$ gcc -c -fPIC math_ops.c -o math_ops.o
gufro@MSI:/tmp$ gcc -shared -o libmathops.so math_ops.o
gufro@MSI:/tmp$ |
```

Compile program yang menggunakan library

```
gufro@MSI:/tmp$ gcc main.c -L. -lmathops -o program_shared
gufro@MSI:/tmp$ |
```

Jalankan dengan LD_LIBRARY_PATH

```
gufro@MSI:/tmp$ LD_LIBRARY_PATH=. ./program_shared
5 + 3 = 8
5 * 3 = 15
gufro@MSI:/tmp$ |
```

Lihat informasi loader



6.1 System Call Differences

```
gufro@MSI:/tmp$ readelf -d program_shared

Dynamic section at offset 0x2da8 contains 28 entries:
   Tag               Type              Name/Value
0x0000000000000001 (NEEDED)         Shared library: [libmathops.so]
0x0000000000000001 (NEEDED)         Shared library: [libc.so.6]
0x000000000000000c (INIT)           0x1000
0x000000000000000d (FINI)           0x11e4
0x0000000000000019 (INIT_ARRAY)     0x3d98
0x000000000000001b (INIT_ARRAYSZ)      8 (bytes)
0x000000000000001a (FINI_ARRAY)     0x3da0
0x000000000000001c (FINI_ARRAYSZ)    8 (bytes)
0x000000006ffffff5 (GNU_HASH)       0x3b0
0x0000000000000005 (STRTAB)         0x4b0
0x0000000000000006 (SYMTAB)         0x3d8
0x000000000000000a (STRSZ)          170 (bytes)
0x000000000000000b (SYMENT)         24 (bytes)
0x0000000000000015 (DEBUG)          0x0
0x0000000000000003 (PLTGOT)         0x3fa8
0x0000000000000002 (PLTRELSZ)       72 (bytes)
0x0000000000000014 (PLTREL)         RELA
0x0000000000000017 (JMPREL)         0x660
0x0000000000000007 (RELA)           0x5a0
0x0000000000000008 (RELASZ)         192 (bytes)
0x0000000000000009 (RELAENT)        24 (bytes)
0x000000000000001e (FLAGS)          BIND_NOW
0x000000006ffffffb (FLAGS_1)        Flags: NOW PIE
0x000000006ffffffe (VERNEED)        0x570
0x000000006fffffff (VERNEEDNUM)     1
0x000000006ffffff0 (VERSYM)         0x55a
0x000000006ffffff9 (RELACOUNT)      3
0x0000000000000000 (NULL)           0x0
gufro@MSI:/tmp$
```

Menjelaskan bahwa aplikasi bergantung pada sistem operasi karena setiap OS memiliki system call berbeda untuk berkomunikasi dengan kernel. Program `os_specific.c` menunjukkan hal ini dengan menggunakan `#ifdef _linux_` dan `#elif _WIN32` agar kode bisa menyesuaikan dengan OS yang digunakan. Pada Linux, program memakai `<unistd.h>` dan `<sys/utsname.h>` untuk menampilkan informasi sistem dan ID proses, sedangkan di Windows menggunakan `<windows.h>` dengan fungsi berbeda. Dari sini terlihat bahwa program yang dibuat untuk Linux belum tentu bisa dijalankan di Windows tanpa perubahan, karena tiap sistem operasi memiliki fungsi dan API sendiri.

Buat file os_specific.c:

```
gufro@MSI:/tmp$ nano os_specific.c
gufro@MSI:/tmp$ |
```

6.2 Binary Format Analysis

membahas analisis format biner pada program di Linux Setelah program C dikompilasi, hasilnya berupa file ELF (Executable and Linkable Format). Perintah file program digunakan untuk melihat jenis dan arsitektur file. readelf -h program menampilkan informasi utama seperti tipe file dan entry point. readelf -S program digunakan untuk melihat bagian-bagian file seperti .text, .data, dan .bss, sedangkan readelf -l program menunjukkan bagaimana program dimuat ke memori. Terakhir, objdump -d program | head -50 digunakan untuk melihat isi program dalam bentuk **kode assembly**, yaitu hasil terjemahan dari kode sumber ke instruksi mesin.

Lihat format executable

```
gufro@MSI:/tmp$ file program
program: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=8c5145e0b63b47095af229368edc342f23076fc2, for GNU/Linux 3.2.0, not stripped
gufro@MSI:/tmp$ |
```

Lihat header ELF

```
gufro@MSI:/tmp$ readelf -h program
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:          ELF64
  Data:            2's complement, little endian
  Version:         1 (current)
  OS/ABI:          UNIX - System V
  ABI Version:     0
  Type:            DYN (Position-Independent Executable file)
  Machine:         Advanced Micro Devices X86-64
  Version:         0x1
  Entry point address: 0x1060
  Start of program headers: 64 (bytes into file)
  Start of section headers: 14072 (bytes into file)
  Flags:          0x0
  Size of this header: 64 (bytes)
  Size of program headers: 56 (bytes)
  Number of program headers: 13
  Size of section headers: 64 (bytes)
  Number of section headers: 31
  Section header string table index: 30
gufro@MSI:/tmp$ |
```

Lihat section headers

```
gufro@MSI:/tmp$ readelf -S program
There are 31 section headers, starting at offset 0x36f8:
```

```
Section Headers:
[Nr] Name           Type              Address            Offset
     Size           EntSize          Flags    Link  Info  Align
[ 0]                  NULL              0000000000000000  0 0 0
     0000000000000000 0000000000000000
[ 1] .interp           PROGBITS          0000000000000318 00000318
     0000000000000000 0000000000000000  A    0    0    1
[ 2] .note.gnu.pr[...] NOTE              0000000000000338 00000338
     0000000000000030 0000000000000000  A    0    0    8
[ 3] .note.gnu.bu[...] NOTE              0000000000000368 00000368
     0000000000000024 0000000000000000  A    0    0    4
[ 4] .note.ABI-tag     NOTE              000000000000038c 0000038c
     0000000000000020 0000000000000000  A    0    0    4
[ 5] .gnu.hash          GNU_HASH          00000000000003b0 000003b0
     0000000000000024 0000000000000000  A    6    0    8
[ 6] .dynsym            DYNSYM            00000000000003d8 000003d8
     00000000000000a8 0000000000000018  A    7    1    8
[ 7] .dynstr            STRTAB            0000000000000480 00000480
     000000000000008f 0000000000000000  A    0    0    1
[ 8] .gnu.version       VERSYM            0000000000000510 00000510
     000000000000000e 0000000000000002  A    6    0    2
[ 9] .gnu.version_r     VERNEED           0000000000000520 00000520
     0000000000000030 0000000000000000  A    7    1    8
[10] .rel.dyn            RELA               0000000000000550 00000550
     00000000000000c0 0000000000000018  A    6    0    8
[11] .rel.plt            RELA               0000000000000610 00000610
     0000000000000018 0000000000000018  AI   6   24    8
[12] .init              PROGBITS           0000000000001000 00001000
     000000000000001b 0000000000000000  AX   0    0    4
[13] .plt                PROGBITS           0000000000001020 00001020
     0000000000000020 0000000000000010  AX   0    0   16
[14] .plt.got            PROGBITS           0000000000001040 00001040
     0000000000000010 0000000000000010  AX   0    0   16
```

```

[14] .plt.got            PROGBITS           0000000000001040 00001040
     0000000000000010 0000000000000010  AX   0    0   16
[15] .plt.sec            PROGBITS           0000000000001050 00001050
     0000000000000010 0000000000000010  AX   0    0   16
[16] .text               PROGBITS           0000000000001060 00001060
     0000000000000171 0000000000000000  AX   0    0   16
[17] .fini                PROGBITS           00000000000011d4 000011d4
     000000000000000d 0000000000000000  AX   0    0    4
[18] .rodata              PROGBITS           0000000000002000 00002000
     000000000000001c 0000000000000000  A    0    0    4
[19] .eh_frame_hdr        PROGBITS           000000000000201c 0000201c
     0000000000000044 0000000000000000  A    0    0    4
[20] .eh_frame            PROGBITS           0000000000002060 00002060
     00000000000000ec 0000000000000000  A    0    0    8
[21] .init_array           INIT_ARRAY         0000000000003db8 00002db8
     0000000000000008 0000000000000008  WA   0    0    8
[22] .fini_array           FINI_ARRAY         0000000000003dc0 00002dc0
     0000000000000008 0000000000000008  WA   0    0    8
[23] .dynamic              DYNAMIC            0000000000003dc8 00002dc8
     00000000000001f0 0000000000000010  WA   7    0    8
[24] .got                  PROGBITS           0000000000003fb8 00002fb8
     0000000000000048 0000000000000008  WA   0    0    8
[25] .data                 PROGBITS           0000000000004000 00003000
     0000000000000010 0000000000000000  WA   0    0    8
[26] .bss                  NOBITS             0000000000004010 00003010
     0000000000000008 0000000000000000  WA   0    0    1
[27] .comment              PROGBITS           0000000000000000 00003010
     000000000000002b 0000000000000001  MS   0    0    1
[28] .symtab               SYMTAB             0000000000000000 00003040
     00000000000003a8 0000000000000018  29  19    8
[29] .strtab               STRTAB             0000000000000000 000033e8
     00000000000001f4 0000000000000000  0    0    1
[30] .shstrtab             STRTAB             0000000000000000 000035dc
     000000000000011a 0000000000000000  0    0    1
Key to Flags:
```

```

000000000000002b 0000000000000001 MS 0 0 1
[28] .symtab SYMTAB 0000000000000000 00003040
000000000000003a8 00000000000000018 29 19 8
[29] .strtab STRTAB 0000000000000000 000033e8
000000000000001f4 0000000000000000 0 0 1
[30] .shstrtab STRTAB 0000000000000000 000035dc
0000000000000011a 0000000000000000 0 0 1
Key to Flags:
W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
D (mbind), l (large), p (processor specific)
gufro@MSI:/tmp$

```

Lihat program headers

```

gufro@MSI:/tmp$ readelf -l program
Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1060
There are 13 program headers, starting at offset 64

Program Headers:
Type           Offset             VirtAddr           PhysAddr
               FileSiz            MemSiz              Flags             Align
PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
               0x00000000000002d8 0x00000000000002d8 R                  0x8
INTERP         0x0000000000000318 0x0000000000000318 0x0000000000000318
               0x000000000000001c 0x000000000000001c R                  0x1
               [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000
               0x0000000000000628 0x0000000000000628 R                  0x1000
LOAD           0x0000000000001000 0x0000000000001000 0x0000000000001000
               0x00000000000001e1 0x00000000000001e1 R E                0x1000
LOAD           0x0000000000002000 0x0000000000002000 0x0000000000002000
               0x000000000000014c 0x000000000000014c R                  0x1000
LOAD           0x0000000000002db8 0x0000000000002db8 0x0000000000002db8
               0x0000000000000258 0x0000000000000260 RW                 0x1000
DYNAMIC        0x0000000000002dc8 0x0000000000002dc8 0x0000000000002dc8
               0x0000000000001f0 0x0000000000001f0 RW                  0x8
NOTE          0x0000000000000338 0x0000000000000338 0x0000000000000338
               0x0000000000000030 0x0000000000000030 R                  0x8
NOTE          0x0000000000000368 0x0000000000000368 0x0000000000000368
               0x0000000000000044 0x0000000000000044 R                  0x4
GNU_PROPERTY   0x0000000000000338 0x0000000000000338 0x0000000000000338
               0x0000000000000030 0x0000000000000030 R                  0x8
GNU_EH_FRAME   0x000000000000201c 0x000000000000201c 0x000000000000201c
               0x0000000000000044 0x0000000000000044 R                  0x4
GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
               0x0000000000000000 0x0000000000000000 RW                  0x10
GNU_RELRO      0x0000000000002db8 0x0000000000002db8 0x0000000000002db8
               0x000000000000248 0x000000000000248 R                  0x1

```

```

GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
               0x0000000000000000 0x0000000000000000 RW                  0x10
GNU_RELRO      0x0000000000002db8 0x0000000000002db8 0x0000000000002db8
               0x000000000000248 0x000000000000248 R                  0x1

Section to Segment mapping:
Segment Sections...
00
01 .interp
02 .interp.note.gnu.property.note.gnu.build-id.note.ABI-tag.gnu.hash.dynsym.dynstr.gnu.version.gnu.version_r.rela.dyn.rela.plt
03 .init.plt.plt.got.plt.sec.text.fini
04 .rodata.eh_frame_hdr.eh_frame
05 .init_array.fini_array.dynamic.got.data.bss
06 .dynamic
07 .note.gnu.property
08 .note.gnu.build-id.note.ABI-tag
09 .note.gnu.property
10 .eh_frame_hdr
11
12 .init_array.fini_array.dynamic.got
gufro@MSI:/tmp$

```


Disassemble

```
gufro@MSI:/tmp$ objdump -d program | head -50
program:      file format elf64-x86-64

Disassembly of section .init:
0000000000001000 <_init>:
1000:    f3 0f 1e fa                endbr64
1004:    48 83 ec 08                sub    $0x8,%rsp
1008:    48 8b 05 d9 2f 00 00      mov    0x2fd9(%rip),%rax        # 3fe8 <__gmon_start___@Base>
100f:    48 85 c0                  test   %rax,%rax
1012:    74 02                     je     1016 <_init+0x16>
1014:    ff d0                     call   *%rax
1016:    48 83 c4 08                add    $0x8,%rsp
101a:    c3                        ret

Disassembly of section .plt:
0000000000001020 <_plt>:
1020:    ff 35 9a 2f 00 00      push   0x2f9a(%rip)            # 3fc0 <_GLOBAL_OFFSET_TABLE_+0x8>
1026:    ff 25 9c 2f 00 00      jmp     *0x2f9c(%rip)          # 3fc8 <_GLOBAL_OFFSET_TABLE_+0x10>
102c:    0f 1f 40 00             nopl    0x0(%rax)
1030:    f3 0f 1e fa                endbr64
1034:    68 00 00 00 00          push    $0x0
1039:    e9 e2 ff ff             jmp     1020 <_init+0x20>
103e:    66 90                     xchg    %ax,%ax

Disassembly of section .plt.got:
0000000000001040 <__cxa_finalize@plt>:
1040:    f3 0f 1e fa                endbr64
1044:    ff 25 ae 2f 00 00      jmp     *0x2fae(%rip)          # 3ff8 <__cxa_finalize@GLIBC_2.2.5>
104a:    66 0f 1f 44 00 00      nopw    0x0(%rax,%rax,1)

Disassembly of section .plt.sec:
0000000000001050 <printf@plt>:
1050:    f3 0f 1e fa                endbr64
1054:    ff 25 76 2f 00 00      jmp     *0x2f76(%rip)          # 3fd0 <printf@GLIBC_2.2.5>
105a:    66 0f 1f 44 00 00      nopw    0x0(%rax,%rax,1)

Disassembly of section .text:
0000000000001060 <_start>:
1060:    f3 0f 1e fa                endbr64
1064:    31 ed                     xor     %ebp,%ebp
1066:    49 89 d1                 mov     %rdx,%r9
1069:    5e                       pop     %rsi
106a:    48 89 e2                 mov     %rsp,%rdx
106d:    48 83 e4 f0              and     $0xfffffffffffffff0,%rsp
gufro@MSI:/tmp$
```

7.1 Kernel Module Sederhana

membahas desain dan implementasi sistem operasi, khususnya dengan membuat kernel module sederhana di Linux. Kernel module adalah potongan kode yang dapat dimasukkan ke kernel tanpa perlu melakukan reboot atau mengompilasi ulang seluruh kernel. Pada praktik ini, dibuat file `hello_module.c` yang berisi fungsi `hello_init()` untuk menampilkan pesan saat modul dimuat dan `hello_exit()` saat modul dilepas. Keduanya menggunakan fungsi `printk()` untuk mencatat pesan ke log kernel. Informasi seperti lisensi, penulis, dan versi ditentukan dengan makro `MODULE_LICENSE`, `MODULE_AUTHOR`, dan `MODULE_VERSION`. Proses

kompilasi dilakukan menggunakan Makefile dengan perintah make, yang memanfaatkan build system kernel Linux. Modul yang telah dikompilasi (hello_module.ko) dapat dimuat dengan sudo insmod hello_module.ko dan dilepas dengan sudo rmmod hello_module. Hasilnya dapat dilihat menggunakan lsmod untuk memastikan modul aktif dan dmesg untuk melihat pesan dari kernel. Praktik ini memperlihatkan bagaimana kernel Linux dapat diperluas secara dinamis melalui modul tambahan.

#Buat file hello_module.c

```
gufro@MSI:/tmp$ nano hello_module.c
gufro@MSI:/tmp$ |
```

```
gufro@MSI:/tmp$ sudo nano Makefile
[sudo] password for gufro:
```

```
gufro@MSI:/tmp$ make
Makefile:3: *** missing separator. Stop.
gufro@MSI:/tmp$ sudo insmod hello_module.ko
insmod: ERROR: could not load module hello_module.ko: No such file or directory
gufro@MSI:/tmp$ lsmod | grep hello
gufro@MSI:/tmp$ dmesg | tail
[ 4517.144653] systemd-journald[42]: Time jumped backwards, rotating.
[ 5268.423993] systemd-journald[42]: Time jumped backwards, rotating.
[ 5465.945193] systemd-journald[42]: Time jumped backwards, rotating.
[ 5803.658819] systemd-journald[42]: Time jumped backwards, rotating.
[ 6118.968663] systemd-journald[42]: Time jumped backwards, rotating.
[ 6308.281440] systemd-journald[42]: Time jumped backwards, rotating.
[ 6988.245897] systemd-journald[42]: Time jumped backwards, rotating.
[ 7101.368913] systemd-journald[42]: Time jumped backwards, rotating.
[ 7344.057283] systemd-journald[42]: Time jumped backwards, rotating.
[ 7547.640712] systemd-journald[42]: Time jumped backwards, rotating.
gufro@MSI:/tmp$ sudo rmmod hello_module
rmmod: ERROR: Module hello_module is not currently loaded
gufro@MSI:/tmp$ dmesg | tail
[ 4517.144653] systemd-journald[42]: Time jumped backwards, rotating.
[ 5268.423993] systemd-journald[42]: Time jumped backwards, rotating.
[ 5465.945193] systemd-journald[42]: Time jumped backwards, rotating.
[ 5803.658819] systemd-journald[42]: Time jumped backwards, rotating.
[ 6118.968663] systemd-journald[42]: Time jumped backwards, rotating.
[ 6308.281440] systemd-journald[42]: Time jumped backwards, rotating.
[ 6988.245897] systemd-journald[42]: Time jumped backwards, rotating.
[ 7101.368913] systemd-journald[42]: Time jumped backwards, rotating.
[ 7344.057283] systemd-journald[42]: Time jumped backwards, rotating.
[ 7547.640712] systemd-journald[42]: Time jumped backwards, rotating.
```

7.2 Monitoring Kernel

membahas cara **memantau kernel Linux** untuk memahami bagaimana sistem operasi bekerja di tingkat inti. Langkah pertama adalah melihat versi kernel dengan `uname -r`, yang menampilkan nomor versi kernel yang sedang digunakan. Perintah `cat /proc/version` menampilkan informasi lebih lengkap, termasuk versi GCC dan waktu kompilasi kernel. Selanjutnya, `sysctl -a | head -20` digunakan untuk melihat berbagai **parameter kernel**, seperti pengaturan memori, jaringan, dan keamanan yang bisa diubah secara dinamis. Perintah `dmesg | tail -30` menampilkan **kernel ring buffer**, yaitu log aktivitas kernel terbaru, misalnya saat perangkat atau modul dimuat. Untuk melihat **daftar modul kernel** yang sedang aktif, digunakan `lsmod`, sedangkan `modinfo ext4` menampilkan informasi detail tentang modul tertentu, seperti versi, lisensi, dan deskripsinya. Praktik ini membantu memahami cara kerja kernel serta bagaimana modul dan parameter berperan dalam sistem Linux.

Lihat informasi kernel

```
gufro@MSI:/tmp$ uname -r
6.6.87.2-microsoft-standard-WSL2
gufro@MSI:/tmp$ cat /proc/version
Linux version 6.6.87.2-microsoft-standard-WSL2 (root@439a258ad544) (gcc (GCC) 11.2.0, GNU ld (GNU Binutils) 2.37) #1 SMP PREEMPT_DYNAMIC Thu Jun 5 18:30:46 UTC 2025
gufro@MSI:/tmp$
```

Lihat parameter kernel

```
gufro@MSI:/tmp$ sysctl -a | head -20
sysctl: permission denied on key 'kernel.cad_pid'
abi.vsyscall32 = 1
debug.exception-trace = 1
debug.kprobes-optimization = 1
dev.cdrom.autoclose = 1
dev.cdrom.autoeject = 0
dev.cdrom.check_media = 0
dev.cdrom.debug = 0
dev.cdrom.info = CD-ROM information, Id: cdrom.c 3.20 2003/12/17
dev.cdrom.info =
dev.cdrom.info = drive name:
dev.cdrom.info = drive speed:
dev.cdrom.info = drive # of slots:
dev.cdrom.info = Can close tray:
dev.cdrom.info = Can open tray:
dev.cdrom.info = Can lock tray:
dev.cdrom.info = Can change speed:
dev.cdrom.info = Can select disk:
dev.cdrom.info = Can read multisession:
dev.cdrom.info = Can read MCN:
dev.cdrom.info = Reports media changed:
gufro@MSI:/tmp$
```

Lihat kernel ring buffer

```
gufro@MSI:/tmp$ dmesg | tail -30
[ 2.715620] systemd-journald[42]: File /var/log/journal/d5bf750c0a2b41aea082d5131c91e4c6/system.journal corrupted or uncleanly shut down, renaming and replacing.
[ 2.893011] ACPI: AC: AC Adapter [AC1] (off-line)
[ 2.944305] ACPI: battery: Slot [BAT1] (battery present)
[ 3.257733] kvm_intel: Using Hyper-V Enlightened VMCS
[ 3.454014] intel_rapl_msr: PL4 support detected.
[ 4.853470] systemd-journald[42]: Time jumped backwards, rotating.
[ 4.870078] systemd-journald[42]: Failed to read journal file /var/log/journal/d5bf750c0a2b41aea082d5131c91e4c6/user-1000.journal for rotation, trying to move it out of the way: Device or resource busy
[ 9.889045] WSL (230) ERROR: CheckConnection: getaddrinfo() failed: -5
[ 34.682658] systemd-journald[42]: Time jumped backwards, rotating.
[ 44.211722] systemd-journald[42]: Time jumped backwards, rotating.
[ 48.414819] hv_balloon: Max. dynamic memory size: 8052 MB
[ 574.888547] mini_init (221): drop_caches: 1
[ 1384.513197] systemd-journald[42]: Time jumped backwards, rotating. (Dropped 1 similar message(s))
[ 1613.783939] systemd-journald[42]: Time jumped backwards, rotating.
[ 1954.361944] systemd-journald[42]: Time jumped backwards, rotating.
[ 2001.713552] systemd-journald[42]: Time jumped backwards, rotating.
[ 3037.513098] systemd-journald[42]: Time jumped backwards, rotating.
[ 3882.574831] TCP: eth0: Driver has suspect GRO implementation, TCP performance may be compromised.
[ 3943.093614] mini_init (221): drop_caches: 1
[ 4030.252870] mini_init (221): drop_caches: 1
[ 4517.144653] systemd-journald[42]: Time jumped backwards, rotating.
[ 5268.423993] systemd-journald[42]: Time jumped backwards, rotating.
[ 5465.945193] systemd-journald[42]: Time jumped backwards, rotating.
[ 5803.658019] systemd-journald[42]: Time jumped backwards, rotating.
[ 6110.960663] systemd-journald[42]: Time jumped backwards, rotating.
[ 6308.281440] systemd-journald[42]: Time jumped backwards, rotating.
[ 6980.245897] systemd-journald[42]: Time jumped backwards, rotating.
[ 7101.368913] systemd-journald[42]: Time jumped backwards, rotating.
[ 7344.057283] systemd-journald[42]: Time jumped backwards, rotating.
[ 7547.640712] systemd-journald[42]: Time jumped backwards, rotating.
gufro@MSI:/tmp$
```

Lihat kernel modules

```
gufro@MSI:/tmp$ lsmod
Module                  Size  Used by
tls                     98304  0
intel_rapl_msr          16384  0
intel_rapl_common       32768  1 intel_rapl_msr
kvm_intel               356352  0
kvm                     970752  1 kvm_intel
irqbypass              12288  1 kvm
crc32c_intel            16384  0
battery                 20480  0
ac                      16384  0
sch_fq_codel            16384  1
configfs                53248  1
autofs4                 45056  0
br_netfilter            28672  0
bridge                  282624  1 br_netfilter
stp                     12288  1 bridge
llc                     12288  2 bridge,stp
ip_tables                28672  0
tun                      53248  0
gufro@MSI:/tmp$
```

Informasi modul tertentu

```
gufro@MSI:/tmp$ modinfo ext4
name: ext4
filename: (builtin)
softdep: pre: crc32c
license: GPL
file: fs/ext4/ext4
description: Fourth Extended Filesystem
author: Remy Card, Stephen Tweedie, Andrew Morton, Andreas Dilger, Theodore Ts'o and others
alias: fs-ext4
alias: ext3
alias: fs-ext3
gufro@MSI:/tmp$
```

8.1 Eksplorasi Struktur Layering

menjelaskan struktur berlapis sistem operasi melalui eksplorasi langsung di Linux. Pada lapisan 1 (Hardware), perintah `lspci`, `lsusb`, dan `cat /proc/cpuinfo` digunakan untuk melihat informasi perangkat keras seperti CPU dan USB. Lapisan 2 (Kernel) diamati dengan `uname -a` dan `cat /proc/sys/kernel/ostype` untuk mengetahui jenis dan versi kernel. Lapisan 3 (System Calls) dipelajari dengan `strace ls`, yang menampilkan panggilan sistem antara program dan kernel. Lapisan 4 (System Programs) menggunakan `ps aux` dan `top` untuk memantau proses yang berjalan. Terakhir, lapisan 5 (User Applications) memakai `which bash` dan `which gcc` untuk melihat lokasi aplikasi pengguna. Praktik ini membantu memahami bagaimana setiap lapisan bekerja sama dalam sistem Linux.

Layer 1: Hardware

```
gufro@MSI:/tmp$ lspci
Command 'lspci' not found, but can be installed with:
sudo apt install pciutils
gufro@MSI:/tmp$ lsusb
Command 'lsusb' not found, but can be installed with:
sudo apt install usbutils
gufro@MSI:/tmp$ cat /proc/cpuinfo
processor       : 0
vendor_id      : GenuineIntel
cpu family     : 6
model          : 186
model name     : Intel(R) Core(TM) 5 120U
stepping       : 3
microcode      : 0xffffffff
cpu MHz        : 2496.000
cache size     : 12288 KB
physical id    : 0
siblings       : 12
core id        : 0
cpu cores      : 6
apicid         : 0
initial apicid: 0
fpu            : yes
fpu_exception  : yes
cpuid level    : 28
wp             : yes
flags          : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdt
scp lm constant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2
apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shado
w ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 xsaves avx
_vnni vmni umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_clear serialize flush_l1d arch_capabilities
vmx flags      : vnni invvpid ept_x_only ept_ad ept_lgb tsc_offset vtptr ept vpid unrestricted_guest ept_mode_based_exec tsc_scaling usr_wait_pa
use
bugs           : spectre_v1 spectre_v2 spec_store_bypass swapgs retbleed eibrs_pbrsb rfds bhi
bogomips       : 4992.00
```

Layer 2: Kernel

```
gufro@MSI:/tmp$ uname -a
Linux MSI 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
gufro@MSI:/tmp$ cat /proc/sys/kernel/ostype
Linux
gufro@MSI:/tmp$
```

Layer 3: System Calls

```
gufro@MSI:/tmp$ strace ls
Command 'strace' not found, but can be installed with:
sudo apt install strace
gufro@MSI:/tmp$
```

Layer 4: System Programs

```
gufro@MSI:/tmp$ ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1   0.0  0.1 22088 12660 ?        Ss   14:35   0:02 /sbin/init
root           2   0.0  0.0   3072  1664 ?        Sl   14:35   0:00 /init
root           6   0.0  0.0   3072  1792 ?        Sl   14:35   0:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-trunc
root          42   0.0  0.2  66820 18752 ?        S<s  14:35   0:01 /usr/lib/systemd/systemd-journald
root          90   0.0  0.0  25264  6016 ?        Ss   14:35   0:00 /usr/lib/systemd/systemd-udevd
systemd+     104   0.0  0.1  21456 12160 ?        Ss   14:35   0:00 /usr/lib/systemd/systemd-resolved
systemd+     105   0.0  0.0  91024  7552 ?        Ssl  14:35   0:00 /usr/lib/systemd/systemd-timesyncd
root         165   0.0  0.0   4236  2560 ?        Ss   14:35   0:00 /usr/sbin/cron -f -P
message+     166   0.0  0.0   9640  4736 ?        Ss   14:35   0:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activ
root         173   0.0  0.1  17968  8448 ?        Ss   14:35   0:00 /usr/lib/systemd/systemd-logind
root         175   0.0  0.1 1756096 12928 ?       Ssl  14:35   0:00 /usr/libexec/wsl-pro-service -vv
root         189   0.0  0.0   3160  1920 hvc0     Ss+  14:35   0:00 /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 vt220
root         197   0.0  0.0   3116  1792 ttyl     Ss+  14:35   0:00 /sbin/agetty -o -p -- \u --noclear - linux
syslog       199   0.0  0.0  222508  5376 ?        Ssl  14:35   0:00 /usr/sbin/rsyslogd -n -iNONE
root         206   0.0  0.2 107032 21888 ?        Ssl  14:35   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown -
root         291   0.0  0.0   3076   896 ?        Ss   14:35   0:00 /init
root         292   0.0  0.0   3092  1152 ?        S   14:35   0:00 /init
gufro        293   0.0  0.0   6204  5248 pts/0    Ss   14:35   0:00 -bash
root         294   0.0  0.0   6820  4608 pts/1    Ss   14:35   0:00 /bin/login -f
gufro        338   0.0  0.1  20308 11136 ?        Ss   14:35   0:00 /usr/lib/systemd/systemd --user
gufro        339   0.0  0.0   21148  3516 ?        S   14:35   0:00 (sd-pam)
gufro        363   0.0  0.0   6072  5248 pts/1    S+   14:35   0:00 -bash
gufro        538   0.0  0.0   2680  1536 pts/0    T   15:08   0:00 ./simple_shell
gufro        656   0.0  0.0  14772  5888 pts/0    T   15:36   0:00 systemctl list-units --type=service
gufro        657   0.0  0.0   3512  2304 pts/0    T   15:36   0:00 less
gufro        667   0.0  0.0  14772  5888 pts/0    T   15:38   0:00 systemctl list-units --type=service
gufro        668   0.0  0.0   3512  2304 pts/0    T   15:38   0:00 less
gufro        676   0.0  0.0  14772  5888 pts/0    T   15:41   0:00 systemctl list-units --type=service
gufro        677   0.0  0.0   3512  2304 pts/0    T   15:41   0:00 less
polkitd      927   0.0  0.0  308164  7424 ?        Ssl  15:43   0:00 /usr/lib/polkit-1/polkitd --no-debug
gufro       1394   0.0  0.0   2680  1152 ?        Ss   16:02   0:00 ./my_daemon
gufro       1703  50.0  0.0   8200  4096 pts/0    R+   17:03   0:00 ps aux
```

```
gufro@MSI:/tmp$ top
top - 17:04:46 up 2:29, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 32 total, 1 running, 20 sleeping, 7 stopped, 0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 7803.2 total, 7064.9 free,  537.3 used,  357.3 buff/cache
MiB Swap: 2048.0 total, 2048.0 free,  0.0 used,  7265.9 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
    1 root        20   0   22088 12660 9076 S  0.0   0.2   0:02.06 systemd
    2 root        20   0   3072   1664 1664 S  0.0   0.0   0:00.00 init-systemd(Ub
    6 root        20   0   3072   1792 1792 S  0.0   0.0   0:00.00 init
   42 root       19  -1   66820 18752 17728 S  0.0   0.2   0:01.45 systemd-journal
   90 root        20   0   25264  6016 4864 S  0.0   0.1   0:00.44 systemd-udevd
  104 systemd+    20   0   21456 12160 10112 S  0.0   0.2   0:00.18 systemd-resolve
  105 systemd+    20   0   91024  7552  6784 S  0.0   0.1   0:00.38 systemd-timesyn
  165 root        20   0   4236   2560 2432 S  0.0   0.0   0:00.04 cron
  166 message+    20   0   9640   4736 4224 S  0.0   0.1   0:00.57 dbus-daemon
  173 root        20   0   17968  8448 7552 S  0.0   0.1   0:00.26 systemd-logind
  175 root        20   0 1756096 12928 10368 S  0.0   0.2   0:00.55 wsl-pro-service
  189 root        20   0   3160   1920 1792 S  0.0   0.0   0:00.01 agetty
  197 root        20   0   3116   1792 1664 S  0.0   0.0   0:00.01 agetty
  199 syslog      20   0  222508  5376 44080 S  0.0   0.1   0:00.27 rsyslogd
  206 root        20   0 107032 21888 12928 S  0.0   0.3   0:00.14 unattended-upgr
  291 root        20   0   3076   896  896 S  0.0   0.0   0:00.00 SessionLeader
  292 root        20   0   3092  1152 1024 S  0.0   0.0   0:00.71 Relay(293)
  293 gufro       20   0   6204  5248 3584 S  0.0   0.1   0:00.47 bash
  294 root        20   0   6820  4608 3840 S  0.0   0.1   0:00.00 login
  338 gufro       20   0  20308 11136 9088 S  0.0   0.1   0:00.15 systemd
  339 gufro       20   0  21148  3516 1792 S  0.0   0.0   0:00.00 (sd-pam)
  363 gufro       20   0   6072  5248 3584 S  0.0   0.1   0:00.01 bash
  538 gufro       20   0   2680  1536 1536 T  0.0   0.0   0:00.00 simple_shell
  656 gufro       20   0  14772  5888 5120 T  0.0   0.1   0:00.00 systemctl
  657 gufro       20   0   3512  2304 2048 T  0.0   0.0   0:00.00 less
  667 gufro       20   0  14772  5888 5120 T  0.0   0.1   0:00.00 systemctl
  668 gufro       20   0   3512  2304 2048 T  0.0   0.0   0:00.00 less
```

Layer 5: User Applications

```
gufro@MSI:/tmp$ which bash
/usr/bin/bash
gufro@MSI:/tmp$ which gcc
/usr/bin/gcc
gufro@MSI:/tmp$ |
```

8.2 Microkernel vs Monolithic

Membahas perbedaan antara **microkernel** dan **monolithic kernel** dengan mengeksplorasi kernel Linux. Linux termasuk **monolithic kernel**, artinya sebagian besar fungsi seperti manajemen memori, sistem file, dan driver dijalankan langsung di dalam kernel. Perintah `ls -lh /boot/vmlinuz-$(uname -r)` digunakan untuk melihat ukuran kernel yang biasanya besar karena banyak fungsi sudah terintegrasi. Perintah `cat /boot/config-$(uname -r) | grep "=y" | wc -l` menghitung fitur yang dibangun langsung (**built-in**), sedangkan `grep "=m"` menghitung fitur sebagai **module**. Terakhir, `lsmod | wc -l` menampilkan jumlah modul aktif. Praktik ini menunjukkan bahwa monolithic kernel lebih besar dan kompleks dibanding microkernel yang lebih kecil dan hanya berisi fungsi dasar.

Lihat ukuran kernel (Monolithic)

```
gufro@MSI:/tmp$ ls -lh /boot/vmlinuz-$(uname -r)
ls: cannot access '/boot/vmlinuz-6.6.87.2-microsoft-standard-WSL2': No such file or directory
gufro@MSI:/tmp$
```

Lihat built-in vs module

```
gufro@MSI:/tmp$ cat /boot/config-$(uname -r) | grep "=y" | wc -l
cat: /boot/config-6.6.87.2-microsoft-standard-WSL2: No such file or directory
0
gufro@MSI:/tmp$ cat /boot/config-$(uname -r) | grep "=m" | wc -l
cat: /boot/config-6.6.87.2-microsoft-standard-WSL2: No such file or directory
0
gufro@MSI:/tmp$
```

Lihat loaded modules

```
gufro@MSI:/tmp$ lsmod | wc -l
19
gufro@MSI:/tmp$
```


8.3 Program - Virtual File System

membahas tentang Virtual File System (VFS) di Linux melalui program sederhana bernama `procfs_read.c`. Program ini menunjukkan bagaimana informasi sistem dapat diakses lewat direktori `/proc`, yang merupakan bagian dari VFS dan berisi data dinamis tentang sistem dan proses. Program membaca beberapa file virtual seperti `/proc/cpuinfo` untuk menampilkan informasi CPU, `/proc/meminfo` untuk menampilkan kondisi memori, dan `/proc/[pid]/status` untuk menampilkan informasi proses yang sedang berjalan. Setiap bagian dibaca menggunakan fungsi `fopen()` dan `fgets()` lalu ditampilkan ke layar. Setelah dikompilasi dengan `gcc -o procfs_read procfs_read.c` dan dijalankan dengan `./procfs_read`, program menampilkan data dari kernel tanpa mengakses perangkat keras secara langsung. Praktik ini menunjukkan bagaimana VFS memungkinkan pengguna dan program untuk membaca informasi sistem seolah-olah dari file biasa, padahal data tersebut berasal dari kernel Linux secara langsung.

Buat file `procfs_read.c`

```
gufro@MSI:/tmp$ nano procfs_read.c
gufro@MSI:/tmp$ gcc -o procfs_read procfs_read.c
gufro@MSI:/tmp$ ./procfs_read
=== CPU Information ===
processor       : 0
vendor_id      : GenuineIntel
cpu family     : 6
model          : 186
model name     : Intel(R) Core(TM) 5 120U
stepping       : 3
microcode      : 0xffffffff
cpu MHz        : 2496.000
cache size     : 12288 KB
physical id    : 0

=== Memory Information ===
MemTotal:      7990480 kB
MemFree:       7241132 kB
MemAvailable:  7450432 kB
Buffers:       8504 kB
Cached:        327096 kB

=== Process Information ===
Name:   procfs_read
Umask:  0022
State:  R (running)
Tgid:   1771
Ngid:   0
Pid:    1771
PPid:   293
TracerPid: 0
Uid:    1000 1000 1000 1000
Gid:    1000 1000 1000 1000
gufro@MSI:/tmp$
```

9.1 Analisis Boot Process

Membahas proses booting sistem operasi Linux, yaitu tahapan dari saat komputer dinyalakan hingga sistem siap digunakan. Langkah pertama menggunakan `dmesg | less` untuk melihat pesan boot dari kernel, seperti deteksi perangkat keras dan proses inisialisasi sistem. Perintah `systemd-analyze` menampilkan waktu total boot, sedangkan `systemd-analyze blame` dan `systemd-analyze critical-chain` menunjukkan urutan layanan yang dijalankan beserta waktu yang dibutuhkan setiap proses, membantu menganalisis bagian mana yang memperlambat booting. Selanjutnya, `cat /boot/grub/grub.cfg | head -50` digunakan untuk melihat konfigurasi GRUB, yaitu bootloader yang bertugas memuat kernel. Terakhir, `lsinitramfs /boot/initrd.img-$(uname -r) | head -20` menampilkan isi `initramfs`, yaitu sistem file sementara yang digunakan kernel saat pertama kali dijalankan untuk mempersiapkan akses ke root filesystem. Praktik ini membantu memahami bagaimana Linux melakukan proses booting secara berlapis dan terstruktur.

Lihat boot messages

```
gufro@MSI:/tmp$ systemd-analyze critical-chain
The time when unit became active or started is printed after the "@" character.
The time the unit took to start is printed after the "+" character.

graphical.target @1.907s
├─multi-user.target @1.906s
│   └─snapd.seeded.service @871ms +339ms
│       └─basic.target @848ms
│           └─sockets.target @848ms
│               └─snapd.socket @664ms +183ms
│                   └─sysinit.target @651ms
│                       └─systemd-resolved.service @500ms +143ms
│                           └─systemd-tmpfiles-setup.service @462ms +34ms
│                               └─systemd-journal-flush.service @390ms +68ms
│                                   └─systemd-remount-fs.service @349ms +22ms
│                                       └─systemd-journald.socket @308ms
│                                           └─.mount @258ms
│                                               └─.slice @258ms

gufro@MSI:/tmp$ |
57ms systemd-udevd.service
48ms polkit.service
45ms systemd-journald.service
45ms dev-hugepages.mount
44ms dev-mqueue.mount
43ms sys-kernel-debug.mount
42ms sys-kernel-tracing.mount
41ms e2scrub_reap.service
41ms dbus.service
35ms user-runtime-dir@1000.service
34ms systemd-tmpfiles-setup.service
```

Lihat GRUB configuration

```
gufro@MSI:/tmp$ cat /boot/grub/grub.cfg | head -50
cat: /boot/grub/grub.cfg: No such file or directory
gufro@MSI:/tmp$ |
```

Lihat initramfs

```
gufro@MSI:/tmp$ lsinitramfs /boot/initrd.img-$(uname -r) | head -20
Command 'lsinitramfs' not found, but can be installed with:
sudo apt install initramfs-tools-core
gufro@MSI:/tmp$ |
```

9.2 Kernel Parameters

membahas **kernel parameters**, yaitu pengaturan yang mengontrol cara kerja kernel saat booting. Parameter ini digunakan untuk mengatur hal seperti debugging, perangkat keras, atau performa sistem. Perintah `cat /proc/cmdline` digunakan untuk melihat parameter yang sedang aktif. Untuk menambah parameter sementara, tekan ‘e’ pada menu GRUB saat boot, lalu tambahkan parameter pada baris kernel dan tekan Ctrl + X untuk melanjutkan. Jika ingin menambahkannya permanen, edit file `/etc/default/grub`, tambahkan parameter di bagian `GRUB_CMDLINE_LINUX_DEFAULT`, lalu jalankan `sudo update-grub`. Praktik ini menunjukkan cara menyesuaikan perilaku kernel Linux sesuai kebutuhan.

Lihat kernel parameters saat boot

```
gufro@MSI:/tmp$ cat /proc/cmdline
initrd=\initrd.img WSL_ROOT_INIT=1 panic=-1 nr_cpus=12 hv_utils.timesync_implicit=1 console=hvc0 debug ptty.legacy_count=0 WSL_ENABLE_CRASH_DUMP=
1
```

9.3 Simple Bootloader (Konsep)

#Buat file `bootloader_concept.asm` (untuk pembelajaran)

10.1 Using strace

Trace system calls

```
gufro@MSI:/tmp$ strace ls
Command 'strace' not found, but can be installed with:
sudo apt install strace
gufro@MSI:/tmp$ |
```

Trace dengan detail

```
gufro@MSI:/tmp$ strace -v ls
Command 'strace' not found, but can be installed with:
sudo apt install strace
gufro@MSI:/tmp$ |
```

Hitung waktu system calls

```
gufro@MSI:/tmp$ strace -c ls
Command 'strace' not found, but can be installed with:
sudo apt install strace
gufro@MSI:/tmp$ |
```

Trace dan simpan ke file

```
gufro@MSI:/tmp$ strace -o trace.txt ls
Command 'strace' not found, but can be installed with:
sudo apt install strace
gufro@MSI:/tmp$ cat trace.txt
cat: trace.txt: No such file or directory
gufro@MSI:/tmp$ |
```

10.2 Using ltrace

Trace library calls

```
gufro@MSI:/tmp$ ltrace ls
Command 'ltrace' not found, but can be installed with:
sudo apt install ltrace
gufro@MSI:/tmp$ |
```

Dengan detail

```
gufro@MSI:/tmp$ ltrace -c ls
Command 'ltrace' not found, but can be installed with:
sudo apt install ltrace
gufro@MSI:/tmp$ |
```

10.3 Using GDB

Buat program dengan bug buggy.c

Compile dengan debug symbols

```
gufro@MSI:/tmp$ nano buggy.c
gufro@MSI:/tmp$ gcc -g -o buggy buggy.c
```

Jalankan dengan GDB

```
gufro@MSI:/tmp$ gdb ./buggy
Command 'gdb' not found, but can be installed with:
sudo apt install gdb
gufro@MSI:/tmp$ |
```

Di dalam GDB:

```
gufro@MSI:/tmp$ break main
-bash: break: only meaningful in a 'for', 'while', or 'until' loop
gufro@MSI:/tmp$ run
Command 'run' not found, did you mean:
  command 'rue' from snap darkdimension-rue (1.0.7)
  command 'bun' from snap bun-js (1.3)
  command 'zrun' from deb moreutils (0.67-1)
  command 'crun' from deb crun (1.12-1)
  command 'srun' from deb slurm-client (23.02.6-1ubuntu2)
  command 'rup' from deb rstat-client (4.0.1-11)
  command 'rn' from deb trn4 (4.0-test77-16)
  command 'zun' from deb python3-zunclient (4.7.0-0ubuntu1)
  command 'runq' from deb exim4-daemon-heavy (4.97-4ubuntu4.3)
  command 'runq' from deb exim4-daemon-light (4.97-4ubuntu4.3)
  command 'runq' from deb sendmail-bin (8.17.2-1)
  command 'grun' from deb grun (0.9.3+git20200303-3)
  command 'runc' from deb runc (1.3.0-0ubuntu2~24.04.1)
See 'snap info <snapname>' for additional versions.
gufro@MSI:/tmp$ next
Command 'next' not found, but can be installed with:
sudo apt install mailutils-mh # version 1:3.16-1build1, or
sudo apt install mmh # version 0.4-6
sudo apt install nmh # version 1.8-1
gufro@MSI:/tmp$ step
Command 'step' not found, but can be installed with:
sudo apt install step
gufro@MSI:/tmp$ print n
Command 'print' not found, but can be installed with:
sudo apt install mailcap
gufro@MSI:/tmp$ continue
-bash: continue: only meaningful in a 'for', 'while', or 'until' loop
gufro@MSI:/tmp$ backtrace
backtrace: command not found
gufro@MSI:/tmp$ quit
Command 'quit' not found, but can be installed with:
sudo snap install quit
```

10.4 Kernel Debugging dengan dmesg

Lihat kernel log

```
gufro@MSI:/tmp$ dmesg
[ 0.000000] Linux version 6.6.87.2-microsoft-standard-WSL2 (root@439a258ad544) (gcc (GCC) 11.2.0, GNU ld (GNU Binutils) 2.37) #1 SMP PREEMPT_
DYNAMIC Thu Jun  5 10:30:46 UTC 2025
[ 0.000000] Command line: initrd=initrd.img WSL_ROOT_INIT=1 panic=1 nr_cpus=12 hv_utils.timesync_implicit=1 console=hvc0 debug=pty.legacy.c
ount=0 WSL_ENABLE_CRASH_DUMP=1
[ 0.000000] KERNEL supported cpus:
[ 0.000000] Intel GenuineIntel
[ 0.000000] AMD AuthenticAMD
[ 0.000000] BIOS-provided physical RAM map:
[ 0.000000] BIOS-e820: [mem 0x0000000000000000-0x00000000000009fff] usable
[ 0.000000] BIOS-e820: [mem 0x0000000000000000-0x00000000000000fff] reserved
[ 0.000000] BIOS-e820: [mem 0x0000000000100000-0x00000000001ffff] ACPI data
[ 0.000000] BIOS-e820: [mem 0x0000000000200000-0x00000000002ffff] usable
[ 0.000000] BIOS-e820: [mem 0x0000000010000000-0x00000000100000fff] usable
[ 0.000000] BIOS-e820: [mem 0x0000000010000000-0x00000000100000fff] usable
[ 0.000000] NX (Execute Disable) protection: active
[ 0.000000] APIC: Static calls initialized
[ 0.000000] DMI not present or invalid.
[ 0.000000] Hypervisor detected: Microsoft Hyper-V
[ 0.000000] Hyper-V: privilege flags low 0xae7f, high 0x3b8830, hints 0x9a4e24, misc 0xe0bed7b2
[ 0.000000] Hyper-V: Nested features: 0x3e0101
[ 0.000000] Hyper-V: LAPIC Timer Frequency: 0xc3500
[ 0.000000] Hyper-V: Using hypercall for remote TLB flush
[ 0.000000] clocksource: hyperv_clocksource_tsc.page: mask: 0xffffffffffff max_cycles: 0x24e6a1710, max_idle_ns: 440795202120 ns
[ 0.000000] tsc: Detected 2495.000 MHz processor
[ 0.000000] e820: update [mem 0x00000000-0x000000fff] usable ==> reserved
[ 0.000000] e820: remove [mem 0x000a0000-0x0000ffff] usable
[ 0.000000] last_pfn = 0xffff000 max_arch_pfn = 0x000000000
[ 0.000000] MTRR map: 5 entries (4 fixed + 1 variable); max 20, built from 8 variable MTRRs
[ 0.000000] x86/PAT: Configuration [0-7]: WB WC UC- UC WB WP UC- WT
[ 0.000111] last_pfn = 0xf8000 max_arch_pfn = 0x400000000
[ 0.000120] Using GB pages for direct mapping
[ 0.000245] RAMDISK: [mem 0x00000000-0x00007ffff]
[ 0.000247] ACPI: Early table checksum verification disabled
[ 0.000251] ACPI: RSDP 0x0000000000000000 000024 (v02 VIRTUAL)
[ 0.000254] ACPI: XSDT 0x0000000000010000 000004 (v01 VIRTUAL MICROSOFT 00000001 MSFT 00000001)
```

Follow kernel log

```
gufro@MSI:/tmp$ dmesg -w
[ 0.000000] Linux version 6.6.87.2-microsoft-standard-WSL2 (root@439a258ad544) (gcc (GCC) 11.2.0, GNU ld (GNU Binutils) 2.37) #1 SMP PREEMPT_
DYNAMIC Thu Jun 5 18:30:46 UTC 2025
[ 0.000000] Command line: initrd=\initrd.img WSL_ROOT_INIT=1 panic=-1 nr_cpus=12 hv_utils.timesync_implicit=1 console=hvc0 debug pty.legacy_c
ount=0 WSL_ENABLE_CRASH_DUMP=1
[ 0.000000] KERNEL supported cpus:
[ 0.000000] Intel GenuineIntel
[ 0.000000] AMD AuthenticAMD
[ 0.000000] BIOS-provided physical RAM map:
[ 0.000000] BIOS-e820: [mem 0x0000000000000000-0x0000000000000fff] usable
[ 0.000000] BIOS-e820: [mem 0x0000000000000000-0x0000000000000fff] reserved
[ 0.000000] BIOS-e820: [mem 0x0000000000100000-0x000000000001ffff] ACPI data
[ 0.000000] BIOS-e820: [mem 0x0000000000200000-0x0000000000f7ffff] usable
[ 0.000000] BIOS-e820: [mem 0x0000000100000000-0x00000001ff3fffff] usable
[ 0.000000] NX (Execute Disable) protection: active
[ 0.000000] APIC: Static calls initialized
[ 0.000000] DMI not present or invalid.
[ 0.000000] Hypervisor detected: Microsoft Hyper-V
[ 0.000000] Hyper-V: privilege flags low 0xae7f, high 0x3b8030, hints 0x9a4e24, misc 0xe0bed7b2
[ 0.000000] Hyper-V: Nested features: 0x3e0101
[ 0.000000] Hyper-V: LAPIC Timer Frequency: 0xc3500
[ 0.000000] Hyper-V: Using hypercall for remote TLB flush
[ 0.000000] clocksource: hyperv_clocksource_tsc_page: mask: 0xffffffffffffff max_cycles: 0x24e6a1710, max_idle_ns: 440795202120 ns
[ 0.000000] clocksource: hyperv_clocksource_msr: mask: 0xffffffffffffff max_cycles: 0x24e6a1710, max_idle_ns: 440795202120 ns
[ 0.000000] tsc: Detected 2496.000 MHz processor
[ 0.000000] e820: update [mem 0x00000000-0x00000fff] usable ==> reserved
[ 0.000000] e820: remove [mem 0x000a0000-0x000fffff] usable
[ 0.000000] last_pfn = 0x1fff400 max_arch_pfn = 0x400000000
[ 0.000000] MTRR map: 5 entries (4 fixed + 1 variable; max 20), built from 8 variable MTRRs
[ 0.000000] x86/PAT: Configuration [0-7]: WB WC UC- UC WB WP UC- WT
[ 0.000000] last_pfn = 0xf8000 max_arch_pfn = 0x400000000
[ 0.000000] Using GB pages for direct mapping
[ 0.000000] RAMDISK: [mem 0x04be4000-0x04e7ffff]
[ 0.000000] ACPI: Early table checksum verification disabled
[ 0.000000] ACPI: RSDP 0x0000000000000000 000024 (v02 VIRTUAL)
```

Filter by level

```
gufro@MSI:/tmp$ dmesg -l err
[ 0.112087] PCI: Fatal: No config space access function found
[ 2.445312] misc dxg: dxgk: dxgkio_is_feature_enabled: Ioctl failed: -22
[ 2.450602] misc dxg: dxgk: dxgkio_query_adapter_info: Ioctl failed: -22
[ 2.451997] misc dxg: dxgk: dxgkio_query_adapter_info: Ioctl failed: -22
[ 2.453265] misc dxg: dxgk: dxgkio_query_adapter_info: Ioctl failed: -22
[ 2.454707] misc dxg: dxgk: dxgkio_query_adapter_info: Ioctl failed: -22
[ 9.889845] WSL (238) ERROR: CheckConnection: getaddrinfo() failed: -5
gufro@MSI:/tmp$ dmesg -l warn
[ 0.005772] Hyper-V: Disabling IBT because of Hyper-V bug
[ 0.146591] PCI: System does not support PCI
[ 0.202758] device-mapper: core: CONFIG_IMA_DISABLE_HTABLE is disabled. Duplicate IMA measurements will not be recorded in the IMA log.
[ 2.305530] pulseaudio[273]: mainfd_create() called without MFD_EXEC or MFD_NOEXEC_SEAL set
[ 2.715620] systemd-journald[42]: File /var/log/journal/d5bf750c0a2b41aea882d5131c91e4c6/system.journal corrupted or uncleanly shut down, ren
aming and replacing.
[ 4.870878] systemd-journald[42]: Failed to read journal file /var/log/journal/d5bf750c0a2b41aea882d5131c91e4c6/user-1000.journal for rotatio
n, trying to move it out of the way: Device or resource busy
[ 3882.574831] TCP: eth0: Driver has suspect GRO implementation, TCP performance may be compromised.
gufro@MSI:/tmp$
```

10.5 Memory Leak Detection dengan Valgrind

Install valgrind

```
gufro@MSI:/tmp$ sudo apt install valgrind
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  gdb libbabeltrace1 libc6-dbg libdebuginfod-common libdebuginfod1t64 libipt2 libsource-highlight-common libsource-highlight4t64
Suggested packages:
  gdb-doc gdbserver valgrind-dbg valgrind-mpi kcachegrind alleyoop valkyrie
The following NEW packages will be installed:
  gdb libbabeltrace1 libc6-dbg libdebuginfod-common libdebuginfod1t64 libipt2 libsource-highlight-common libsource-highlight4t64 valgrind
0 upgraded, 9 newly installed, 0 to remove and 49 not upgraded.
Need to get 26.9 MB of archives.
After this operation, 105 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libdebuginfod-common all 0.190-1.1ubuntu0.1 [14.6 kB]
Ign:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libdebuginfod-common all 0.190-1.1ubuntu0.1
Get:2 http://archive.ubuntu.com/ubuntu noble/main amd64 libbabeltrace1 amd64 1.5.11-3build3 [164 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libdebuginfod1t64 amd64 0.190-1.1ubuntu0.1 [17.1 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble/main amd64 libipt2 amd64 2.0.6-1build1 [45.7 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble/main amd64 libsource-highlight-common all 3.1.9-4.3build1 [64.2 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/main amd64 libsource-highlight4t64 amd64 3.1.9-4.3build1 [258 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble/main amd64 gdb amd64 15.0.50.20240403-0ubuntu1 [4010 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libc6-dbg amd64 2.39-0ubuntu8.6 [7458 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble/main amd64 valgrind amd64 1:3.22.0-0ubuntu3 [14.9 MB]
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libdebuginfod-common all 0.190-1.1ubuntu0.1 [14.6 kB]
Fetched 26.9 MB in 2min 21s (191 kB/s)
Preconfiguring packages ...
Selecting previously unselected package libdebuginfod-common.
(Reading database ... 48892 files and directories currently installed.)
Preparing to unpack .../0-libdebuginfod-common_0.190-1.1ubuntu0.1_all.deb ...
Unpacking libdebuginfod-common (0.190-1.1ubuntu0.1) ...
Selecting previously unselected package libbabeltrace1:amd64.
Preparing to unpack .../1-libbabeltrace1_1.5.11-3build3_amd64.deb ...
Unpacking libbabeltrace1:amd64 (1.5.11-3build3)
```

Buat program dengan memory leak

```
gufro@MSI:/tmp$ cat > memleak.c << 'EOF'
> #include <stdlib.h>
int main() {
    int *ptr = (int*)malloc(100 * sizeof(int));
    // Forgot to free(ptr)
return 0;
}
EOF
gufro@MSI:/tmp$ gcc -g -o memleak memleak.c
gufro@MSI:/tmp$
```

Jalankan dengan valgrind

```
gufro@MSI:/tmp$ valgrind --leak-check=full ./memleak
==2197== Memcheck, a memory error detector
==2197== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==2197== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==2197== Command: ./memleak
==2197==
==2197== HEAP SUMMARY:
==2197==   in use at exit: 400 bytes in 1 blocks
==2197== total heap usage: 1 allocs, 0 frees, 400 bytes allocated
==2197==
==2197== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==2197==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==2197==    by 0x10915E: main (memleak.c:3)
==2197==
==2197== LEAK SUMMARY:
==2197==   definitely lost: 400 bytes in 1 blocks
==2197==   indirectly lost: 0 bytes in 0 blocks
==2197==   possibly lost: 0 bytes in 0 blocks
==2197==   still reachable: 0 bytes in 0 blocks
==2197==   suppressed: 0 bytes in 0 blocks
==2197==
==2197== For lists of detected and suppressed errors, rerun with: -s
==2197== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
gufro@MSI:/tmp$
```


10.6 Performance Analysis

CPU profiling dengan perf

```
gufro@MSI:/tmp$ sudo apt install linux-tools-common linux-tools-generic
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  hwdetect libnuma1 libpci3 libtraceevent1 linux-tools-6.8.0-85 linux-tools-6.8.0-85-generic
The following NEW packages will be installed:
  hwdetect libnuma1 libpci3 libtraceevent1 linux-tools-6.8.0-85 linux-tools-6.8.0-85-generic linux-tools-common linux-tools-generic
0 upgraded, 8 newly installed, 0 to remove and 49 not upgraded.
Need to get 6340 kB of archives.
After this operation, 21.0 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 libnuma1 amd64 2.0.18-1build1 [23.3 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/main amd64 libpci3 amd64 1:3.10.0-2build1 [36.5 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libtraceevent1 amd64 1:1.8.2-1ubuntu2.1 [55.3 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble/main amd64 hwdetect all 0.379-1 [29.1 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 linux-tools-common all 6.8.0-85.85 [762 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 linux-tools-6.8.0-85 amd64 6.8.0-85.85 [5421 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 linux-tools-6.8.0-85-generic amd64 6.8.0-85.85 [1884 B]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 linux-tools-generic amd64 6.8.0-85.85 [10.8 kB]
Fetched 6340 kB in 28s (228 kB/s)
Selecting previously unselected package libnuma1:amd64.
(Reading database ... 58974 files and directories currently installed.)
Preparing to unpack .../0-libnuma1_2.0.18-1build1_amd64.deb ...
Unpacking libnuma1:amd64 (2.0.18-1build1) ...
Selecting previously unselected package libpci3:amd64.
Preparing to unpack .../1-libpci3_1:3.10.0-2build1_amd64.deb ...
Unpacking libpci3:amd64 (1:3.10.0-2build1) ...
Selecting previously unselected package libtraceevent1:amd64.
Preparing to unpack .../2-libtraceevent1_1:1.8.2-1ubuntu2.1_amd64.deb ...
Unpacking libtraceevent1:amd64 (1:1.8.2-1ubuntu2.1) ...
```

Monitor system calls

```
gufro@MSI:/tmp$ sudo perf trace ls
WARNING: perf not found for kernel 6.6.87.2-microsoft

You may need to install the following packages for this specific kernel:
  linux-tools-6.6.87.2-microsoft-standard-WSL2
  linux-cloud-tools-6.6.87.2-microsoft-standard-WSL2

You may also want to install one of the following packages to keep up to date:
  linux-tools-standard-WSL2
  linux-cloud-tools-standard-WSL2
gufro@MSI:/tmp$ |
```

Top-like interface

```
gufro@MSI:/tmp$ sudo perf top
WARNING: perf not found for kernel 6.6.87.2-microsoft

You may need to install the following packages for this specific kernel:
  linux-tools-6.6.87.2-microsoft-standard-WSL2
  linux-cloud-tools-6.6.87.2-microsoft-standard-WSL2

You may also want to install one of the following packages to keep up to date:
  linux-tools-standard-WSL2
  linux-cloud-tools-standard-WSL2
gufro@MSI:/tmp$ |
```

PENUTUP

Kesimpulan

Dari seluruh rangkaian praktikum Sistem Operasi, dapat disimpulkan bahwa kegiatan ini memberikan pemahaman mendalam tentang bagaimana sistem operasi bekerja dalam mengatur sumber daya komputer, proses, memori, dan layanan sistem. Mahasiswa mempelajari berbagai konsep penting seperti manajemen proses, system call, linking, layanan systemd, kernel module, hingga struktur sistem operasi Linux secara menyeluruh. Melalui penerapan langsung dalam bentuk program C, peserta praktikum dapat memahami bagaimana teori sistem operasi diterapkan pada lingkungan nyata serta bagaimana interaksi antara pengguna, sistem, dan kernel terjadi secara dinamis.

Saran

Praktikum ini sangat membantu dalam memperkuat pemahaman teori sistem operasi. Namun, agar lebih optimal, disarankan untuk menambahkan materi lanjutan seperti manajemen memori tingkat lanjut, penjadwalan proses (*process scheduling*), serta keamanan kernel (*kernel security*). Selain itu, pemberian proyek integratif di akhir pertemuan akan membantu mahasiswa menggabungkan seluruh konsep yang telah dipelajari menjadi satu sistem yang utuh dan fungsional.

